

CS2006 – Operating Systems

# Chapter 9: Main Memory

---

## Complete Exam Preparation Notes

*Paging · TLB · EAT · Hierarchical Page Tables · Address Binding · MMU · Fragmentation · Swapping*

Instructor: Anaum Hamid | Spring 2026

# Table of Contents

---

1. Background & Overview
2. Address Binding
3. Logical vs Physical Address Space & MMU
4. Contiguous Memory Allocation
5. Fragmentation (Internal & External)
6. Swapping
7. Paging – Concepts & Diagrams
8. Page Table & Its Implementation
9. Translation Lookaside Buffer (TLB)
10. Effective Access Time (EAT)
11. Memory Protection in Paging
12. Shared Pages
13. Structure of the Page Table (Hierarchical / Hashed / Inverted)
14. Solved Numericals
15. Practice Questions – Theory & Numericals

# 1. Background & Overview

---

Memory management is the core OS function that controls how processes are loaded, placed, and tracked in main memory. Only the CPU registers and main memory are directly accessible by the CPU.

## Key Concepts

- A program must be loaded into main memory before it can execute.
- CPU registers are accessed in one clock cycle or less.
- Main memory access takes many cycles, causing a stall — the cache exists to bridge this gap.
- The OS must protect memory: each process must only access its own memory space.

## Base and Limit Registers

A pair of hardware registers defines each process's legal address space:

- Base register: holds the starting physical address of the process.
- Limit register: holds the length of the logical address space.

Every memory access in user mode is checked:  $\text{base} \leq \text{address} < \text{base} + \text{limit}$ . A violation triggers a trap to the OS (protection fault).

□ Exam Tip: Know that loading base/limit registers is a PRIVILEGED instruction — user programs cannot modify them.

## 2. Address Binding

Address binding is the process of mapping a program's symbolic addresses to actual memory addresses. It can occur at three stages:

| Stage          | When?                               | Key Feature                                                 |
|----------------|-------------------------------------|-------------------------------------------------------------|
| Compile Time   | If memory location known beforehand | Generates absolute code; must recompile if location changes |
| Load Time      | If location unknown at compile time | Generates relocatable code; loader resolves addresses       |
| Execution Time | Process can move during execution   | Requires hardware support (base/limit registers, MMU)       |

### Multi-Step Processing of a User Program

Source Code → Compiler → Object Module → Linker (with other objects & libraries) → Load Module → Loader → Memory

### Static vs Dynamic Linking

| Static Linking                                     | Dynamic Linking                                 |
|----------------------------------------------------|-------------------------------------------------|
| System libraries combined with binary at link time | Linking postponed until execution time          |
| Every executable contains its own copy             | Small stub code locates memory-resident library |
| Larger binary size                                 | Smaller binary; library shared among processes  |
| No run-time overhead                               | Small run-time overhead on first call           |

□ Exam Tip: Know all three binding stages with examples. Execution-time binding is the basis of virtual memory.

### 3. Logical vs Physical Address Space & MMU

#### Logical Address (Virtual Address)

Generated by the CPU during instruction execution. The running process only ever sees logical addresses. The full set of logical addresses a program can use is its logical address space.

#### Physical Address

The actual address in RAM sent to the memory unit. The set of all physical addresses is the physical address space.

| Logical Address                       | Physical Address                       |
|---------------------------------------|----------------------------------------|
| Generated by CPU                      | Seen by the memory hardware            |
| Same as physical at compile/load time | Differs from logical at execution time |
| Controlled by the process             | Controlled by the OS & hardware (MMU)  |

#### Memory Management Unit (MMU)

The MMU is a hardware device that translates logical addresses to physical addresses at runtime. The simplest scheme adds the value in the relocation register (formerly base register) to every logical address generated by a user process.

$$\text{Physical Address} = \text{Logical Address} + \text{Relocation Register Value}$$

*Simple MMU translation scheme*

- The user program never sees real physical addresses; it only works with logical addresses. The OS maintains the mappings.

## 4. Contiguous Memory Allocation

In contiguous allocation, each process occupies one single contiguous section of memory. Main memory is split into:

- Low memory: reserved for the OS (with interrupt vector).
- High memory: allocated to user processes.

### Memory Allocation Strategies for Holes (Dynamic Storage Allocation)

When a process arrives it is allocated from a "hole" — a block of free memory. Three strategies exist:

| Algorithm | How it works                                     | Evaluation                                                               |
|-----------|--------------------------------------------------|--------------------------------------------------------------------------|
| First-Fit | Allocate the first hole that is large enough     | Fastest; may leave many small holes at the start                         |
| Best-Fit  | Allocate the smallest hole that fits the process | Best memory utilization (in theory); slower; creates tiny leftover holes |
| Worst-Fit | Allocate the largest available hole              | Slowest and worst utilization; leaves large leftover holes               |

□ First-Fit and Best-Fit both outperform Worst-Fit in speed and storage utilization.

### Worked Example (from Lecture)

Memory partitions: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB

Processes: P1=212 KB, P2=417 KB, P3=112 KB, P4=426 KB

**First-Fit:** 212→500K | 417→600K | 112→288K(leftover) | 426 WAITS

**Best-Fit:** 212→300K | 417→500K | 112→200K | 426→600K ✓ All fit!

**Worst-Fit:** 212→600K | 417→500K | 112→388K | 426 WAITS

□ Exam Tip: In this classic example Best-Fit wins. Always show workings step-by-step in exams.

## 5. Fragmentation

| External Fragmentation                               | Internal Fragmentation                            |
|------------------------------------------------------|---------------------------------------------------|
| Total free memory is enough but it is NOT contiguous | Allocated block is slightly larger than requested |
| Occurs in variable-partition schemes                 | Occurs in fixed/paged allocation                  |
| 50% rule: ~1/3 of memory can be wasted               | Worst case: page size – 1 byte wasted per process |
| Solution: Compaction (requires dynamic relocation)   | Solution: Choose appropriate page/block size      |

### Compaction

Compaction shuffles all process memory to eliminate external fragmentation by placing all free memory into one contiguous block. It is only possible if relocation is dynamic and done at execution time. Compaction is expensive (heavy I/O burden).

### Internal Fragmentation Calculation (Paging)

$$\text{Internal Fragmentation} = \text{Page Size} - (\text{Process Size} \bmod \text{Page Size})$$

$$\text{Average} = \text{Page Size} / 2$$

### Worked Example

Page size = 2,048 bytes, Process size = 72,766 bytes

$72,766 \div 2,048 = 35$  full pages + 1,086 bytes remaining

Internal Fragmentation =  $2,048 - 1,086 = 962$  bytes

## 6. Swapping

---

Swapping allows a process to be temporarily moved out of main memory to a backing store (swap space on disk) and later brought back to continue execution. This lets the total size of all processes exceed physical memory.

### Swapping Process

1. A process that is waiting (e.g., blocked on I/O) is swapped OUT to backing store.
2. Another process is swapped IN to use the freed memory.
3. The original process is later swapped back in to resume execution.

### Key Points

- Context switch time is dominated by swap time, which depends on process size.
- The process must be swapped back to the SAME memory location if relocation binding was at load time, OR any location if execution-time binding is used.
- Swapping with paging means only individual pages are swapped, not the whole process (more efficient).

□ Exam Tip: Swapping is different from virtual memory (demand paging). In basic swapping, the ENTIRE process is moved; in demand paging, only individual pages are moved.

## 7. Paging – Concepts & Diagrams

Paging allows a process's physical address space to be NON-CONTIGUOUS, solving the external fragmentation problem.

### Core Idea

- Physical memory divided into fixed-size blocks called FRAMES.
- Logical memory divided into same-sized blocks called PAGES.
- Page size is always a power of 2 (512 B to 16 MB).
- The OS maintains a PAGE TABLE for every process to map pages to frames.

### Address Translation

Every logical address is split into two parts:

**Logical Address = [ Page Number (p) | Page Offset (d) ]**

*If logical space =  $2^m$  and page size =  $2^n \rightarrow p$  has  $(m-n)$  bits,  $d$  has  $n$  bits*

Translation steps:

4. Extract page number  $p$  from the logical address (high-order bits).
5. Look up  $p$  in the page table to get frame number  $f$ .
6. Physical address =  $f$  concatenated with the offset  $d$ .

**Physical Address = [ Frame Number (f) | Page Offset (d) ]**

*Offset  $d$  is UNCHANGED; only the page number is replaced by the frame number*

### Paging Diagram (Text Representation)

| Paging Hardware Flow                       |
|--------------------------------------------|
| CPU generates Logical Address: [ p   d ]   |
| ↓                                          |
| Page Table (indexed by p) → Frame Number f |
| ↓                                          |
| Physical Address: [ f   d ] → Main Memory  |

### Bit Calculation Rules

| Given                                                          | Formula                                                                                             |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Logical address space = $2^m$ bytes, page size = $2^n$ bytes   | Offset bits = $n$ ; Page number bits = $m - n$                                                      |
| Physical address space = $2^k$ bytes, frame size = $2^n$ bytes | Offset bits = $n$ ; Frame number bits = $k - n$                                                     |
| $N$ pages in logical space, $M$ frames in physical space       | Logical addr bits = $\log_2 N + \text{offset bits}$ ;<br>Physical = $\log_2 M + \text{offset bits}$ |

□ Exam Tip: The offset  $d$  is ALWAYS the same number of bits as  $\log_2$  (page size). If page size = 1 KB =  $2^{10}$ , offset = 10 bits.

## 8. Page Table & Its Implementation

---

The page table stores one entry per page in the logical address space. Each entry contains the frame number and protection/status bits.

### Storage of the Page Table

- Stored in MAIN MEMORY.
- Page-Table Base Register (PTBR): holds the physical address of the page table.
- Page-Table Length Register (PTLR): holds the number of entries (size) of the page table.
- Every memory access now needs TWO memory references: one for the page table lookup + one for the actual data.

□ The two-access problem is solved by the TLB (Translation Lookaside Buffer) — covered in Section 9.

## 9. Translation Lookaside Buffer (TLB)

The TLB is a small, fast hardware cache (associative memory) that stores recently used page-to-frame mappings. It performs parallel lookup, avoiding the need to access the page table in main memory for common pages.

### TLB Operation

7. CPU generates logical address with page number  $p$ .
8.  $p$  is looked up simultaneously in ALL TLB entries (parallel/associative search).
9. **TLB Hit:** Frame number found instantly  $\rightarrow$  physical address formed  $\rightarrow$  1 memory access.
10. **TLB Miss:** Frame number NOT in TLB  $\rightarrow$  access page table in memory  $\rightarrow$  2 memory accesses  $\rightarrow$  load entry into TLB for future use.

| TLB Feature                       | Details                                                                           |
|-----------------------------------|-----------------------------------------------------------------------------------|
| Size                              | Typically 64–1024 entries                                                         |
| ASIDs (Address Space Identifiers) | Identifies which process each entry belongs to; avoids flushing on context switch |
| TLB Miss Handling                 | Load missing entry from page table; apply replacement policy                      |
| Wired Entries                     | Some entries permanently fixed (kernel mappings)                                  |
| Context Switch (no ASID)          | Entire TLB must be flushed — expensive                                            |

### TLB Flow Diagram

| TLB Address Translation Flow                                                          |
|---------------------------------------------------------------------------------------|
| Logical Address [ $p$   $d$ ]                                                         |
| ↓                                                                                     |
| TLB (Associative Memory) — Hit —→ Frame $f$ → Physical Address [ $f$   $d$ ] → Memory |
|                                                                                       |
| Miss                                                                                  |
| ↓                                                                                     |
| Page Table in Memory → Frame $f$ → Physical Address [ $f$   $d$ ] → Memory            |
| + Load ( $p$ , $f$ ) into TLB for next time                                           |

## 10. Effective Access Time (EAT)

EAT accounts for both TLB hits and misses to compute the average memory access time.

### EAT Formula

$$\text{EAT} = \alpha \times (t_{\text{TLB}} + t_{\text{mem}}) + (1 - \alpha) \times (t_{\text{TLB}} + 2 \times t_{\text{mem}})$$

$\alpha = \text{TLB hit ratio}$  |  $t_{\text{TLB}} = \text{TLB access time}$  |  $t_{\text{mem}} = \text{memory access time}$

When  $t_{\text{TLB}} \approx 0$  (often assumed in problems):

$$\text{EAT} = \alpha \times t_{\text{mem}} + (1 - \alpha) \times 2 \times t_{\text{mem}}$$

*Simplified formula (TLB access time ignored)*

### Worked Examples

#### Example 1 (from lecture) – $\alpha = 80\%$ , memory = 100 ns

$$\begin{aligned}\text{EAT} &= 0.80 \times 100 + 0.20 \times (100 + 100) \\ &= 80 + 40 = 120 \text{ ns}\end{aligned}$$

#### Example 2 – $\alpha = 99\%$ , memory = 100 ns

$$\begin{aligned}\text{EAT} &= 0.99 \times 100 + 0.01 \times 200 \\ &= 99 + 2 = 101 \text{ ns}\end{aligned}$$

#### Example 3 (lecture numerical) – memory = 200 ns, $\alpha = 75\%$ , $t_{\text{TLB}} = 0$

Without TLB (page table in memory):

$$\text{Access time} = 200 + 200 = 400 \text{ ns} \quad (\text{page table} + \text{data})$$

With TLB ( $\alpha = 0.75$ ,  $t_{\text{TLB}} = 0$ ):

$$\begin{aligned}\text{EAT} &= 0.75 \times 200 + 0.25 \times 400 \\ &= 150 + 100 = 250 \text{ ns}\end{aligned}$$

□ Exam Tip: Always check whether the problem includes TLB access time. If  $t_{\text{TLB}} = 0$ , use the simplified formula. On a HIT: 1 mem access for data. On a MISS: 1 for page table + 1 for data = 2 mem accesses.

## 11. Memory Protection in Paging

Memory protection in paging is achieved through bits stored in each page table entry:

| Bit              | Purpose                                                           |
|------------------|-------------------------------------------------------------------|
| Protection bit   | Read-only OR Read-Write access for the frame                      |
| Execute-only bit | Prevents data pages from being executed (prevents code injection) |
| Valid bit (v)    | Page IS in the process's logical address space — legal access     |
| Invalid bit (i)  | Page is NOT in address space — illegal; triggers a trap           |

### Valid-Invalid Bit Mechanism

Each page table entry has a valid (v) or invalid (i) bit. If a process tries to access a page marked invalid, the OS generates a page fault (or protection violation). The Page-Table Length Register (PTLR) also limits the range of accessible page numbers.

### Example – Valid-Invalid Bit Table

| Page Number | Frame / Status                                  |
|-------------|-------------------------------------------------|
| 0           | Frame 4 – Valid (v)                             |
| 1           | Frame 7 – Valid (v)                             |
| 2           | Frame 2 – Valid (v)                             |
| 3           | Frame — – Invalid (i) ← Access triggers OS trap |
| 4           | Frame — – Invalid (i)                           |

□ Exam Tip: Know the difference between 'protection bit' (read/write/execute permissions) and 'valid-invalid bit' (whether page is in address space). Both are stored per page table entry.

## 12. Shared Pages

---

Paging allows multiple processes to share physical frames, saving memory:

### Shared (Reentrant) Code

- Read-only code (e.g., C standard library, text editors) can be shared.
- All processes that use the library map the SAME frame(s) containing it.
- Shared pages must appear at the same logical address in all sharing processes.

### Private Code & Data

- Each process keeps its own copy of modifiable data (stack, heap).
- Private pages can appear at any logical address.

### Copy-on-Write (COW)

- After `fork()`, parent and child initially share the same frames.
- When either process writes to a shared page, the OS creates a private copy.
- This avoids unnecessary copying of pages that are only read.

## 13. Structure of the Page Table

For a 32-bit address space with a 4 KB page size, a simple flat page table would have  $2^{20} \approx 1$  million entries. At 4 bytes/entry that is 4 MB per process — too large to keep contiguously in memory. Three solutions exist:

### A) Hierarchical (Multi-Level) Page Tables

The page table itself is paged. The most common variant is the two-level page table.

#### Two-Level Page Table (32-bit example, 1 KB page)

Logical address (32 bits, 1 KB page → 10-bit offset):

[ p1 (12 bits) | p2 (10 bits) | offset (10 bits) ]  
*p1 = outer page table index | p2 = inner page table index | offset = byte within page*

11. p1 indexes into the outer (1st-level) page table → gives address of inner page table.
12. p2 indexes into the inner (2nd-level) page table → gives frame number f.
13. Physical address = [ f | offset ].

#### Three-Level Page Table (64-bit)

Even two-level is insufficient for 64-bit address spaces. A third level (or more) is added. Each additional level requires an extra memory access per translation, making TLB efficiency critical.

#### Hierarchical Paging Diagram

| Two-Level Page Table Translation                                    |
|---------------------------------------------------------------------|
| Logical Address: [ p1   p2   d ]                                    |
| ↓                                                                   |
| Outer Page Table (indexed by p1) → base address of Inner Page Table |
| ↓                                                                   |
| Inner Page Table (indexed by p2) → Frame Number f                   |
| ↓                                                                   |
| Physical Address: [ f   d ]                                         |

### B) Hashed Page Tables

Common for address spaces larger than 32 bits. The virtual page number is hashed to an index in a hash table. Each slot contains a chain of entries (for collision handling). Each chain entry holds: (virtual page number, frame number, next pointer).

□ Clustered page tables are a 64-bit variant where each entry covers multiple pages (e.g., 16), useful for sparse address spaces.

### C) Inverted Page Tables

Instead of one page table per process (tracking all logical pages), there is ONE global page table with one entry per physical frame.

| Regular Page Table                   | Inverted Page Table                             |
|--------------------------------------|-------------------------------------------------|
| One table per process                | One global table for all processes              |
| Entry per logical page               | Entry per physical frame                        |
| Size $\propto$ logical address space | Size $\propto$ physical memory (fixed, smaller) |
| Fast lookup by index                 | Requires search (use hash table for speed)      |
| Shared pages: easy                   | Shared pages: difficult to represent            |

Each entry: (process ID, virtual page number). A hash table limits search to 1–2 comparisons. TLB is essential to keep performance acceptable.

## 14. Solved Numericals

### Numerical 1 – Page Number & Offset (1 KB page size)

Question: For a 1 KB page size, find page number and offset for each address:

a) 3085   b) 42095   c) 215201   d) 650000   e) 2000001

Method:  $1 \text{ KB} = 1024 = 2^{10} \rightarrow$  offset bits = 10  $\rightarrow$  divide address by 1024

$$\text{Page Number} = \lfloor \text{Address} / \text{Page Size} \rfloor \quad \text{Offset} = \text{Address mod Page Size}$$

| Address | Page Number ( $\lfloor \div 1024 \rfloor$ ) | Offset (mod 1024) |
|---------|---------------------------------------------|-------------------|
| 3085    | 3                                           | 13                |
| 42095   | 41                                          | 111               |
| 215201  | 210                                         | 161               |
| 650000  | 634                                         | 784               |
| 2000001 | 1953                                        | 129               |

Verification for 3085:  $3085 = 3 \times 1024 + 13$  ✓

### Numerical 2 – Bits for Logical and Physical Address

Question: Logical address space = 32 pages, 1024 words/page; Physical = 16 frames. How many bits are needed in the logical and physical addresses?

Logical address space =  $32 \text{ pages} \times 1024 \text{ words} = 2^5 \times 2^{10} = 2^{15}$   
 $\rightarrow$  15 bits for logical address (5 bits page# + 10 bits offset)  
Physical address space =  $16 \text{ frames} \times 1024 \text{ words} = 2^4 \times 2^{10} = 2^{14}$   
 $\rightarrow$  14 bits for physical address (4 bits frame# + 10 bits offset)

### Numerical 3 – Logical to Physical Address (16-bit system)

(From handout Example 1)

Given: Logical space = 16 bits, Physical space = 14 bits, Page size = 1 KB =  $2^{10}$  bytes

Logical address = 0x1A5C, Page table: Page 0x6  $\rightarrow$  Frame 0x2A

Step 1: Offset bits = 10 (page size =  $2^{10}$ )  
Page number bits =  $16 - 10 = 6$  bits

Step 2: 0x1A5C = 0001 1010 0101 1100 (binary, 16 bits)  
Page number = top 6 bits = 000110 = 0x6

```
Offset      = bottom 10 bits = 10 0101 1100 = 0x25C
```

Step 3: Page table lookup: Page 0x6 → Frame 0x2A  
Frame 0x2A = 10 1010 (binary)

Step 4: Physical Address = Frame || Offset  
= 10 1010 10 0101 1100  
= 1010 1010 0101 1100  
= 0xAA5C ✓

## Numerical 4 – Logical to Physical Address (20-bit system)

(From handout Example 2)

Given: Logical space = 20 bits, Physical space = 18 bits, Page size = 4 KB =  $2^{12}$  bytes

Logical address = 0x3F2B9, Page table: Page 0x3F → Frame 0x1C

Step 1: Offset bits = 12 (page size =  $2^{12}$ )  
Page number bits = 20 - 12 = 8 bits

Step 2: 0x3F2B9 = 0011 1111 0010 1011 1001 (binary, 20 bits)  
Page number = top 8 bits = 0011 1111 = 0x3F  
Offset = bottom 12 bits = 0010 1011 1001 = 0x2B9

Step 3: Page table lookup: Page 0x3F → Frame 0x1C  
Frame 0x1C = 01 1100 (binary, 6 bits)

Step 4: Physical Address = 01 1100 || 0010 1011 1001  
= 01 1100 0010 1011 1001  
= 0x1C2B9 ✓

## Numerical 5 – EAT with TLB

Given: Memory access = 200 ns, TLB access = 0 ns, hit ratio  $\alpha$  = 75%

Without TLB: 200 (page table) + 200 (data) = 400 ns

With TLB: EAT =  $0.75 \times 200 + 0.25 \times 400$   
= 150 + 100 = 250 ns

## Numerical 6 – Two-Level Page Table Address Decomposition

Given: 32-bit logical address space, page size = 4 KB ( $2^{12}$ ), two-level paging where inner page tables have  $2^{10} = 1024$  entries.

Offset bits = 12 (page size = 4 KB =  $2^{12}$ )  
Inner PT bits = 10 (inner PT size =  $2^{10}$  entries)  
Outer PT bits = 32 - 12 - 10 = 10 bits

Logical address format: [ p1 (10) | p2 (10) | d (12) ]

Memory accesses per reference: 3 (outer PT + inner PT + data)

# 15. Practice Questions

---

## Part A – Theory / Reasoning Questions

**Q1. What is address binding? Describe all three types with examples.**

**Ans: Address binding maps symbolic program addresses to actual memory addresses. Three types:**

Compile-time: absolute code generated if location known; requires recompilation on change.

Load-time: relocatable code if location unknown at compile time; loader resolves addresses.

Execution-time: binding delayed until runtime; process can be moved; needs hardware (MMU).

**Q2. Differentiate between logical and physical address spaces.**

**Ans: Logical (virtual) address: generated by CPU; what the process sees. Physical address: actual RAM location seen by memory hardware.**

They are identical at compile/load-time binding but differ at execution-time binding. MMU performs the translation.

**Q3. What is the MMU? How does the simple relocation register scheme work?**

**Ans: MMU (Memory Management Unit) is a hardware device that translates logical to physical addresses at runtime.**

Simple scheme: Physical address = Logical address + Relocation register value.

User processes never see physical addresses; the OS sets the relocation register on each context switch.

**Q4. Compare First-Fit, Best-Fit, and Worst-Fit memory allocation algorithms.**

**Ans: First-Fit: allocate first hole large enough — fast but leaves small fragments at start.**

Best-Fit: allocate smallest sufficient hole — good utilization but slow; leaves tiny holes.

Worst-Fit: allocate largest hole — poor performance; leaves large leftover holes.

First-Fit and Best-Fit outperform Worst-Fit in both speed and storage utilization.

**Q5. Explain internal and external fragmentation with examples.**

**Ans: External fragmentation: enough total free memory exists but it is not contiguous (variable partition).**

Internal fragmentation: allocated block is slightly larger than requested; wasted space is internal to the block (paging/fixed partitions).

Example: 962 bytes internal fragmentation when 72,766-byte process runs on 2,048-byte pages.

**Q6. Why is the TLB needed? What happens on a TLB hit vs. TLB miss?**

**Ans: Without TLB, every memory reference needs 2 memory accesses (page table + data) — slow.**

TLB hit: page found in TLB → only 1 memory access (data). TLB miss: access page table → 2 memory accesses → load entry into TLB.  
TLB dramatically reduces average access time via high hit ratios (often >99%).

**Q7. What is the Valid-Invalid bit in a page table? Why is it important?**

**Ans: A 1-bit field per page table entry. Valid (v): page is in the process's legal address space. Invalid (i): page is not accessible; any access triggers a trap to the OS.**

Importance: provides memory protection; prevents a process from accessing memory outside its allocated space.

**Q8. Why was hierarchical paging introduced? Explain two-level paging.**

**Ans: A flat page table for a 32-bit address space with 4 KB pages needs  $2^{20}$  entries  $\times$  4 bytes = 4 MB per process — too large to keep contiguously.**

Two-level paging pages the page table itself. The outer table fits in one page; inner tables are created only when needed. The logical address is split into [p1 | p2 | offset].

**Q9. Compare inverted and regular page tables.**

**Ans: Regular: one table per process; one entry per logical page; size grows with address space.**

Inverted: one global table; one entry per physical frame; size = physical memory (fixed, smaller).  
Inverted reduces memory usage but complicates lookup (need search/hash); shared pages are harder to represent.

**Q10. What is Copy-on-Write (COW)? How does it relate to paging?**

**Ans: After fork(), parent and child share the same physical frames (mapped read-only).**

When either process writes to a shared page, the OS copies that page before allowing the write. COW avoids unnecessary page copies for data that is only read, saving time and memory.

## Part B – Numerical Practice Questions

**Q1.** A system has a logical address space of 64 pages with 512 bytes per page, mapped onto 32 physical frames. How many bits are needed for logical and physical addresses?

**Ans: Page size = 512 =  $2^9$  → offset = 9 bits**

Logical: 64 pages =  $2^6$  → page bits = 6 → total = 6 + 9 = 15 bits

Physical: 32 frames =  $2^5$  → frame bits = 5 → total = 5 + 9 = 14 bits

**Q2.** Page size = 2 KB. Find page number and offset for logical address 8500.

**Ans: 2 KB = 2048 =  $2^{11}$  → offset bits = 11**

Page number =  $\lfloor 8500 / 2048 \rfloor = 4$  ( $4 \times 2048 = 8192$ )

Offset =  $8500 - 8192 = 308$

**Q3.** Logical address space =  $2^{32}$  bytes, page size = 4 KB. How many entries are in the page table (flat)? If each entry = 4 bytes, how much memory does the page table use?

**Ans: Offset bits = 12 (4 KB =  $2^{12}$ ) → Page number bits =  $32 - 12 = 20$**

Number of entries =  $2^{20} = 1,048,576$  entries

Memory =  $1,048,576 \times 4 = 4,194,304$  bytes = 4 MB per process

**Q4.** EAT: TLB hit ratio = 90%, memory access time = 120 ns, TLB access time = 0. Calculate EAT.

**Ans: EAT =  $0.90 \times 120 + 0.10 \times (120 + 120)$**

=  $108 + 24 = 132$  ns

**Q5.** EAT: memory = 150 ns, TLB hit ratio = 95%, TLB access time = 5 ns. Calculate EAT.

**Ans: Hit: 5 ns (TLB) + 150 ns (data) = 155 ns**

Miss: 5 ns (TLB) + 150 ns (page table) + 150 ns (data) = 305 ns

EAT =  $0.95 \times 155 + 0.05 \times 305 = 147.25 + 15.25 = 162.5$  ns

**Q6.** A paging system: logical space = 18 bits, page size = 4 KB. Frame 0x5 is assigned to page 0x3. Translate logical address 0x3A20.

**Ans: Page size = 4 KB =  $2^{12}$  → offset bits = 12**

Page bits =  $18 - 12 = 6$

0x3A20 = 0011 1010 0010 0000

Page# = top 6 bits = 001110 = 0xE — this is NOT page 0x3, so look up your own page table (Adjust: if logical = 0x3A20 = 15,392, page =  $\lfloor 15392/4096 \rfloor = 3$ , offset =  $15392 \bmod 4096 = 3104 = 0xC20$ )

Frame 5 → Physical =  $(5 \times 4096) + 3104 = 20480 + 3104 = 23584 = 0x5C20$

**Q7. Memory partitions: 200 KB, 400 KB, 300 KB, 100 KB, 500 KB. Processes: 150 KB, 350 KB, 200 KB, 400 KB (in order). Show allocation using Best-Fit and First-Fit.**

Ans – First-Fit:

150→200K | 350→400K | 200→300K | 400→500K ✓ All allocated

Ans – Best-Fit:

150→200K (exact fit) | 350→400K (50K waste) | 200→300K (100K waste) | 400→500K (100K waste) ✓

Both algorithms can fit all processes here; Best-Fit has slightly better residual holes.

**Q8. Two-level page table: 32-bit logical address, 4 KB page size, inner page tables cover  $2^{10}$  entries. Write the logical address format and state how many memory accesses per translation.**

**Ans: offset = 12 bits (4 KB), inner = 10 bits, outer =  $32 - 12 - 10 = 10$  bits**

Format: [ p1 (10) | p2 (10) | d (12) ]

Memory accesses: 3 (outer PT + inner PT + data); TLB reduces this to 1 on a hit

## Quick Reference Summary

| Formula / Concept                                              | Description                              |
|----------------------------------------------------------------|------------------------------------------|
| Offset bits = $\log_2$ (page size)                             | Number of bits for page offset           |
| Page bits = total logical bits – offset bits                   | Number of bits for page number           |
| Frame bits = total physical bits – offset bits                 | Number of bits for frame number          |
| $EAT = \alpha \times t\_mem + (1-\alpha) \times 2t\_mem$       | Simplified EAT (TLB access $\approx 0$ ) |
| $EAT = \alpha(t\_TLB + t\_mem) + (1-\alpha)(t\_TLB + 2t\_mem)$ | Full EAT formula                         |
| Internal Frag = Page Size – (process mod page size)            | Wasted space per process                 |
| Physical Addr = [Frame#    Offset]                             | Paging address construction              |
| Page# = $\lfloor \text{Address} / \text{Page Size} \rfloor$    | Page number extraction                   |
| Offset = Address mod Page Size                                 | Offset extraction                        |
| Flat PT size = entries $\times$ entry_bytes                    | Memory for flat page table               |

| Term                   | One-Line Definition                                            |
|------------------------|----------------------------------------------------------------|
| MMU                    | Hardware that translates logical to physical addresses         |
| TLB                    | Fast cache for recent page→frame mappings (associative memory) |
| PTBR                   | Register holding base address of current page table            |
| PTLR                   | Register holding size (length) of current page table           |
| ASID                   | Per-TLB-entry identifier avoids flushing on context switch     |
| COW                    | Copy-on-Write: share frames after fork; copy only on write     |
| External Fragmentation | Free memory exists but is non-contiguous                       |
| Internal Fragmentation | Wasted space inside an allocated block/page                    |
| Compaction             | Moving processes to merge free holes (fixes external frag)     |
| Inverted PT            | One global table, one entry per physical frame                 |
| Hierarchical PT        | Page table is itself paged (2-level, 3-level)                  |
| Hashed PT              | Virtual page# hashed to find frame; used for >32-bit spaces    |